Smart Robotics Project

# Air Hockey


Edoardo Pottocar
272638@studenti.unimore.it

Matteo Bulgarelli
276511@studenti.unimore.it

Giorgia Bertacchini
274372@studenti.unimore.it

# Project Idea

Our reference: Robot Air Hockey Challenge

## Robot
## Franka Emika
## Panda

- Software: ROS & Python 3.8
- Camera-based vision system
- Real-time puck tracking
- Future puck position prediction

# System Setup

## GAZEBO | Physics-based robot simulation

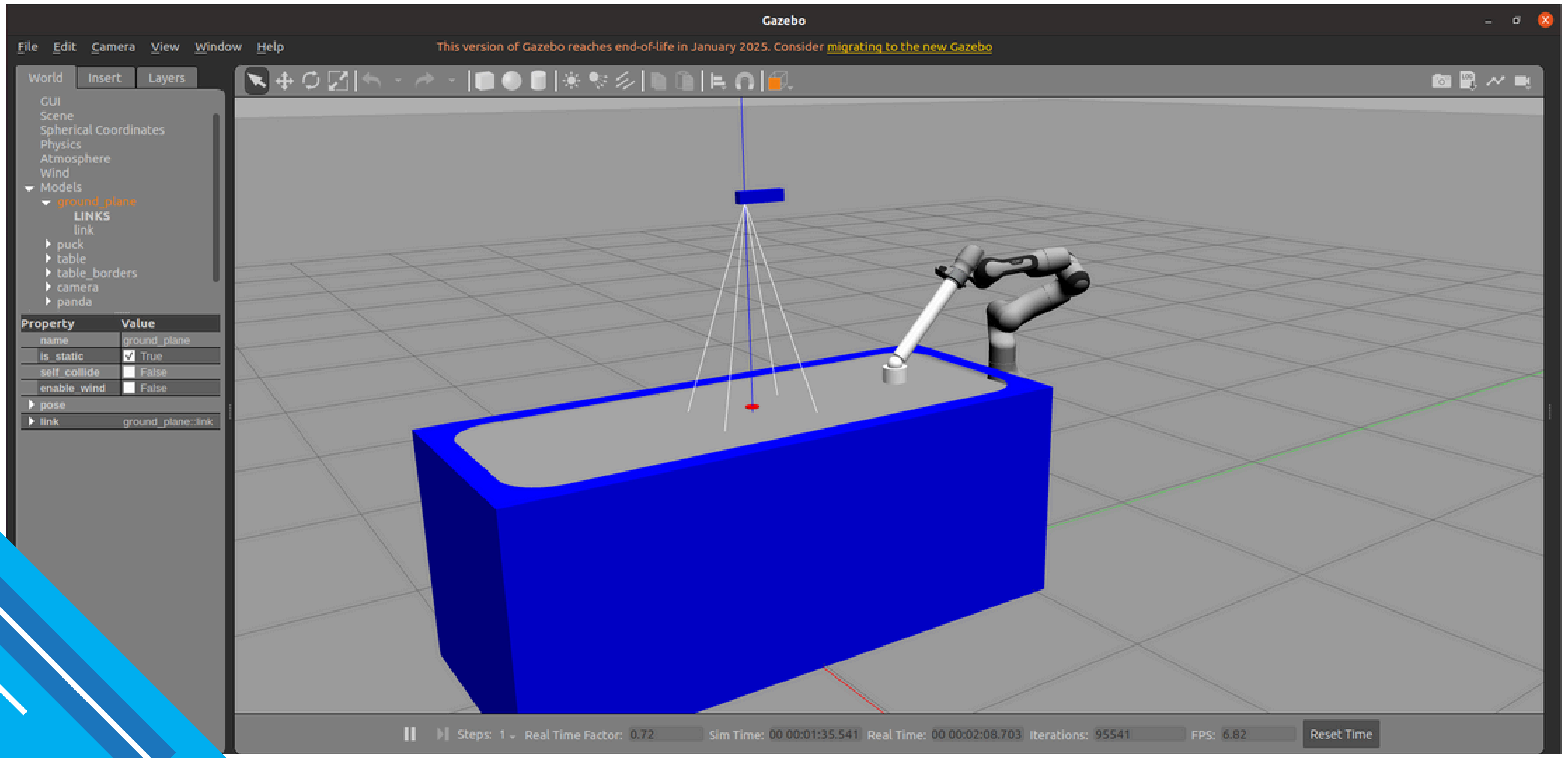
Hardware safety guaranteed

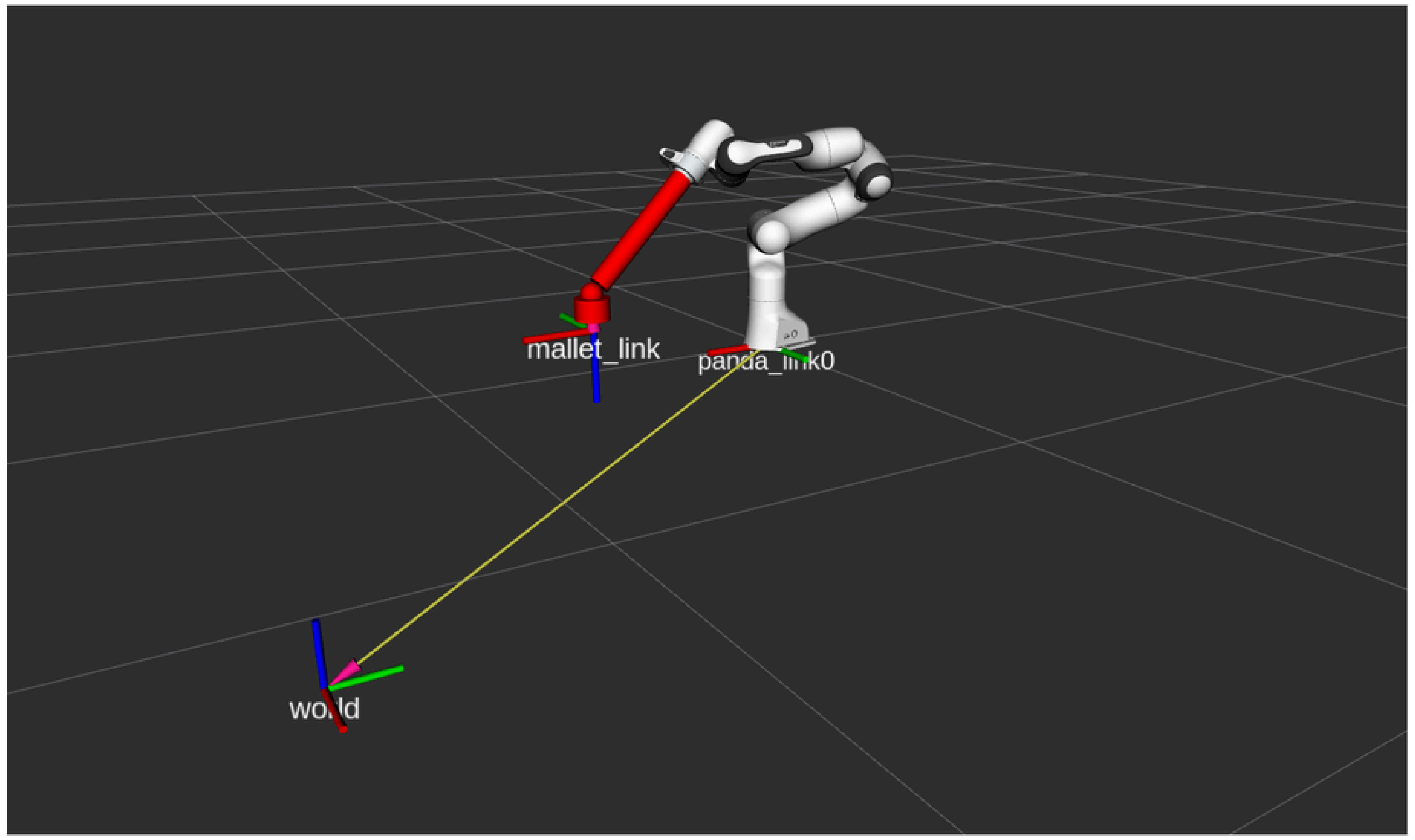
Usefull for prototyping

## RVIZ | ROS visualization tool

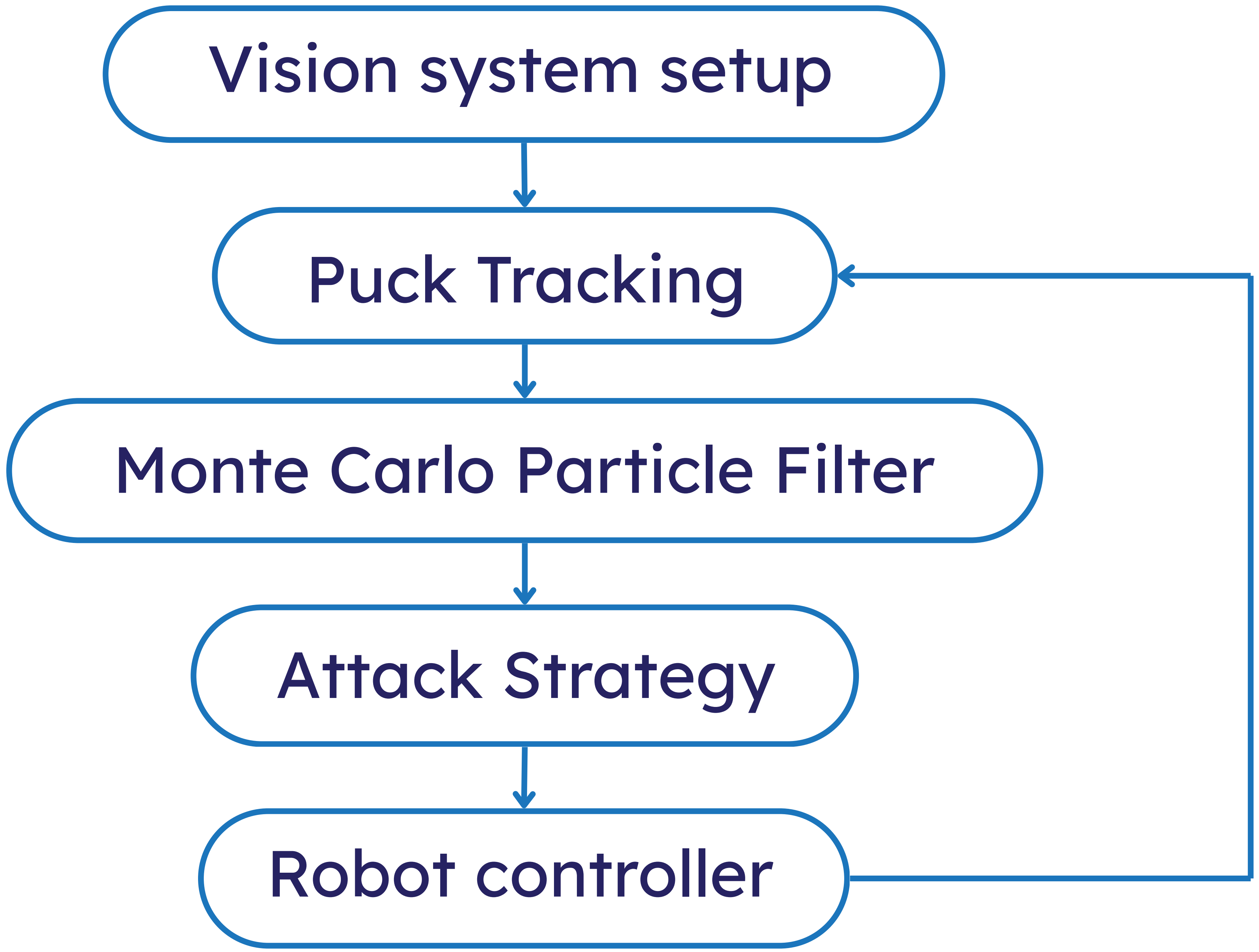
Visualizes robot states and sensor data

Can issue movement commands



# Pipeline


Vision system setup
Puck Tracking
Monte Carlo Particle Filter
Attack Strategy
Robot controller

# 0 Vision system setup

Our OpenCV-based pipeline

**Barrel Distortion correction**

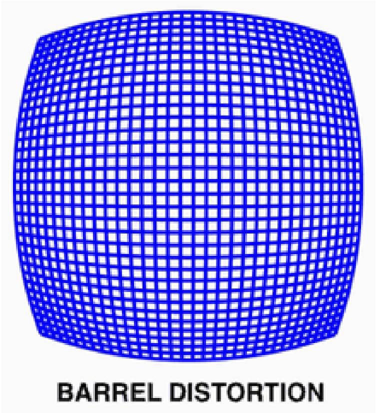



**Table corner detection pipeline**

HSV white masking
↓
Canny edge detection
↓
Hough transform

**Homography computation**

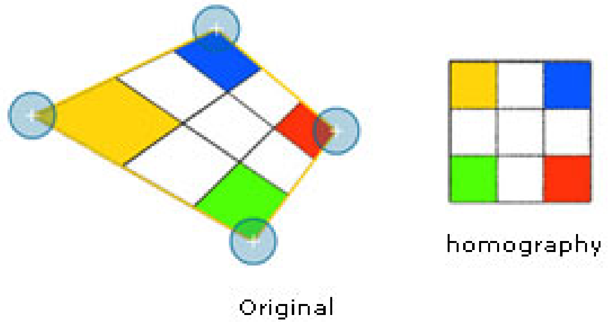
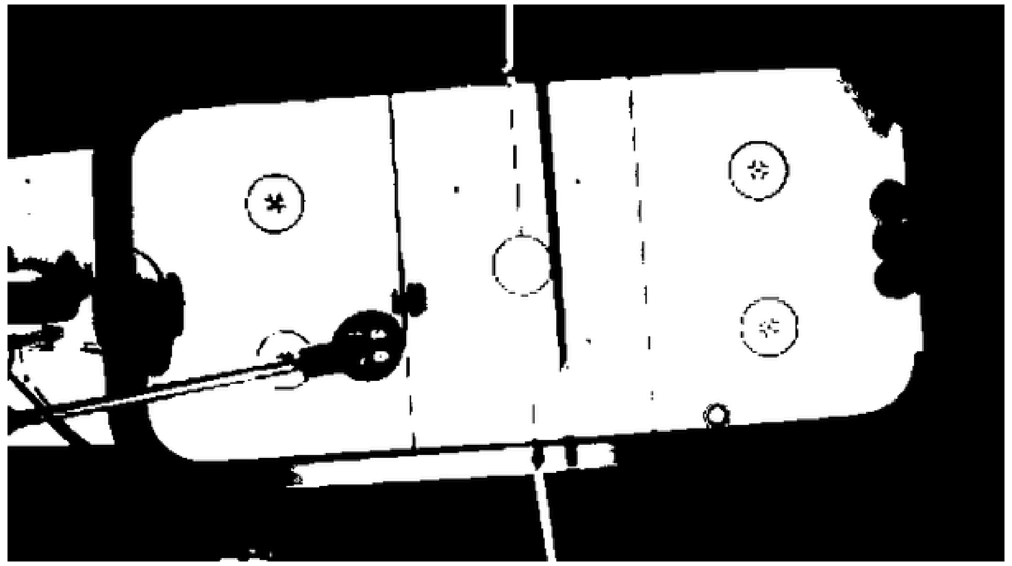



Table without barrel distortion



HSV white masking

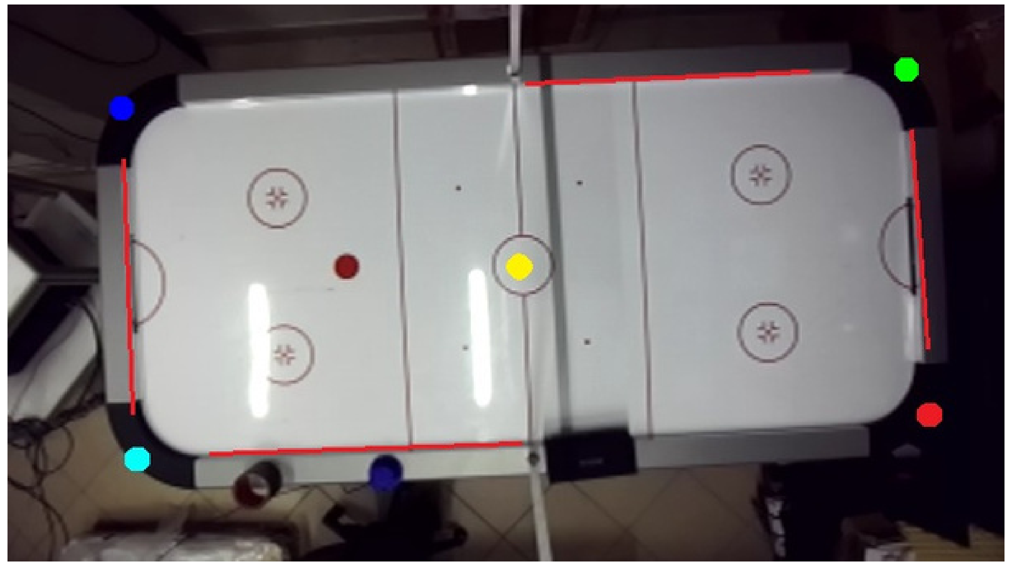


Table corner markers

# 1 Puck Tracking

Puck detection

Our OpenCV-based pipeline

**Puck Detection**

HSV red masking
↓
Circle detection using connected components
↓
Puck center extraction

**pixel-meter Conversion**

Homography-based coordinate conversion

Image-to-world mapping

2

# Monte Carlo Particle Filter

**Initialization**

Generate N particles starting from a “pretrained” filter

**Predict**

Move particles based on physics/motion, adding some random non-gaussian noise to account for non-linearity

**Update**

Adjust weights based on new puck coordinates measurements.

**Exploit**

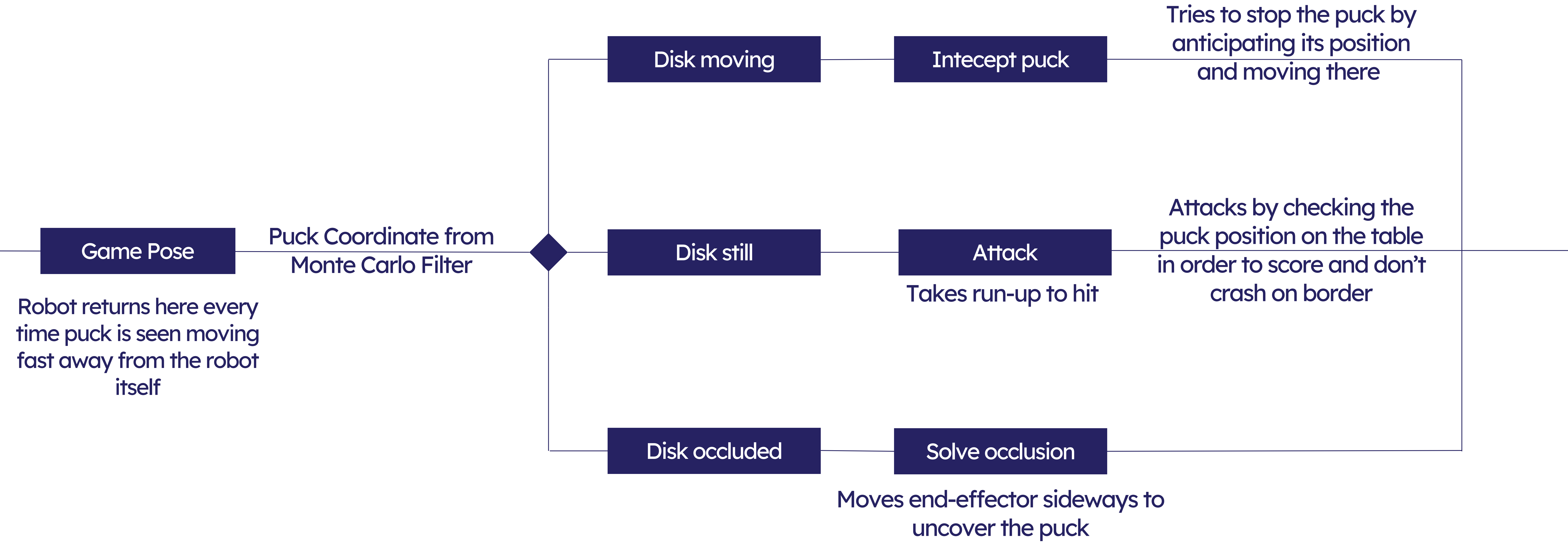
Calculate the "Best Estimate"

**Resample**

Focus the cloud for the next time step.

3

# Attack Strategy



# 4 Robot controller

MoveIt

Our initial control: **Motion Planning Framework**

Limitation: high latency

ROS-based motion planning framework

Trajectory planning and execution

Integrated with RViz for interaction

Recommended: **Cartesian Pose Controller**

Real-time end-effector control in Cartesian space

Commands position and orientation directly (x, y, z, R)

Operates at high control frequency

# Future Improvements

- Integrate an impedance controller

  to enable compliant and safe interaction with the environment and humans

- Explore advanced prediction methods

  Reinforcement Learning (RL)
  LSTM networks

- Migrate from ROS 1 to ROS 2

  improved real-time performance and reliability

Smart Robotics Project

# Thank you

Link GitHub